🚀 Guia Rápido - Implementação Gemini Embedding 2 com Supabase pgvector

Implementações completas para **Arth Executive** e **ChefIA** com busca semântica e personalização.

📦 Arquivos Criados

Arth Executive - Memory Manager

Plain Text

```
/home/ubuntu/arth_embedding/
├── memory_manager.py           # Implementação principal (16KB)
├── schema.sql                  # Schema do banco de dados (7.1KB)
├── README.md                   # Documentação completa (9.6KB)
├── example_integration.py      # Exemplo de integração FastAPI (8.7KB)
├── requirements.txt            # Dependências Python
└── .env.example               # Variáveis de ambiente
```

ChefIA - Recipe Search Engine

Plain Text

```
/home/ubuntu/chefia_embedding/
├── recipe_search.py            # Implementação principal (21KB)
├── schema.sql                  # Schema do banco de dados (13KB)
├── README.md                   # Documentação completa (14KB)
├── example_integration.py      # Exemplo de integração FastAPI (12KB)
├── requirements.txt            # Dependências Python
└── .env.example               # Variáveis de ambiente
```

⚡ Início Rápido

1️⃣ Clonar Repositórios

Bash

```bash
# Arth Executive
git clone https://github.com/Artificiallltda/arth-executive.git
cd arth-executive
```

```
# ChefIA
git clone https://github.com/Artificiallltda/bot_ChefIA.git
cd bot_ChefIA
```

2️⃣ Copiar Arquivos

Bash

```bash
# Para Arth Executive
cp -r /home/ubuntu/arth_embedding/* ./src/

# Para ChefIA
cp -r /home/ubuntu/chefia_embedding/* ./src/
```

3️⃣ Instalar Dependências

Bash

```bash
# Ambos os projetos
pip install -r requirements.txt
```

4️⃣ Configurar Variáveis de Ambiente

Bash

```bash
# Copiar template
cp .env.example .env

# Editar .env com suas credenciais
# GEMINI_API_KEY=sua_chave_aqui
# SUPABASE_URL=seu_url_aqui
# SUPABASE_KEY=sua_chave_aqui
```

5️⃣ Executar Schema SQL

No Supabase Dashboard:

1. Ir para **SQL Editor**
2. Criar nova query
3. Copiar conteúdo de `schema.sql`
4. Executar

Ou via CLI:

```bash
supabase db push < schema.sql
```

6️⃣ Testar Integração

```bash
# Arth Executive
python example_integration.py
# Acesso em http://localhost:8000

# ChefIA
python example_integration.py
# Acesso em http://localhost:8001
```

🎯 Arth Executive - Memory Manager

Funcionalidades Principais

Funcionalidade	Descrição
Armazenamento Semântico	Vetoriza cada mensagem com Gemini Embedding 2 (768 dims)
Busca Contextual	Recupera conversas similares à mensagem atual
Injeção de Contexto	Formata automaticamente contexto para o agente
Gerenciamento de Sessões	Agrupa conversas por sessão
Limpeza Automática	Remove mensagens antigas
Row Level Security	Cada usuário vê apenas suas conversas

Uso Básico

```python
```

```python
from memory_manager import MemoryManager

# Inicializar
manager = MemoryManager()

# Armazenar mensagem
manager.store_message(
    user_id="user_123",
    session_id="session_456",
    role="user",
    content="Quero criar um dashboard"
)

# Recuperar contexto para agente
context = manager.get_context_for_agent(
    user_id="user_123",
    current_message="Como faço para visualizar dados?",
    session_id="session_456"
)

# Usar no agente
agent_response = your_agent(message, context)
```

Endpoints FastAPI Disponíveis

Plain Text

```
POST   /chat                              # Chat com memória semântica
GET    /session/{session_id}/summary      # Resumo da sessão
GET    /session/{session_id}/context      # Contexto para query
POST   /session/{session_id}/clear-old    # Limpar mensagens antigas
GET    /health                            # Verificar saúde
```

🍳 ChefIA - Recipe Search Engine

Funcionalidades Principais

Funcionalidade	Descrição
Busca Semântica	Busca receitas por significado
Personalização	Recomendações por perfil gastronômico

Filtros Inteligentes	Alergias, preferências, tempo de preparo
Avaliações	Sistema de ratings e comentários
Favoritos	Marque receitas e obtenha similares
Trending	Receitas mais populares
Row Level Security	Dados privados por usuário

Uso Básico

Python

```python
from recipe_search import RecipeSearchEngine, UserGastronomicProfile, DietaryPr

# Inicializar
engine = RecipeSearchEngine()

# Criar perfil
profile = UserGastronomicProfile(
    user_id="user_123",
    dietary_preferences=[DietaryPreference.VEGETARIAN],
    favorite_cuisines=[CuisineType.ITALIAN],
    allergies=["peanuts"],
    cooking_time_preference=45,
    spice_level=6
)
engine.save_user_profile(profile)

# Buscar receitas personalizadas
results = engine.personalize_recommendations(
    query="receita rápida",
    user_profile=profile,
    limit=5
)

# Avaliar receita
engine.rate_recipe(
    user_id="user_123",
    recipe_id="recipe_001",
    rating=5,
    comment="Adorei!"
)
```

Endpoints FastAPI Disponíveis

```
POST   /search                    # Buscar receitas
POST   /profile                   # Criar/atualizar perfil
GET    /profile/{user_id}         # Obter perfil
POST   /rate                      # Avaliar receita
GET    /trending                  # Receitas em tendência
POST   /favorites                 # Adicionar favorito
GET    /recommendations/{user_id} # Recomendações
GET    /health                    # Verificar saúde
```

🔐 Segurança

Variáveis de Ambiente

```bash
# Nunca commitar .env
echo ".env" >> .gitignore

# Usar chaves com escopo limitado
GEMINI_API_KEY=...        # Chave com escopo limitado
SUPABASE_KEY=...          # Chave anon (não service key)
```

Row Level Security

Todas as tabelas têm RLS habilitado. Usuários só acessam seus dados:

```sql
-- Automático: usuários só veem suas mensagens
SELECT * FROM conversation_messages
WHERE user_id = auth.uid()::text
```

📊 Modelos de Dados

Arth Executive - Tabelas

Tabela	Descrição

conversation_messages	Mensagens com embeddings (768 dims)
conversation_sessions	Agrupa mensagens em sessões
memory_usage_stats	Estatísticas de uso

ChefIA - Tabelas

Tabela	Descrição
recipes	Receitas com embeddings (768 dims)
user_gastronomic_profiles	Perfil de preferências
recipe_ratings	Avaliações de usuários
recipe_search_history	Histórico de buscas
recipe_favorites	Receitas favoritas

🔍 Busca Semântica

Como Funciona

1. **Embedding**: Texto → Vetor 768-dimensional (Gemini Embedding 2)
2. **Armazenamento**: Vetor armazenado em pgvector
3. **Busca**: Similaridade cosseno entre vetores
4. **Índice**: HNSW para busca rápida

Exemplo de Query SQL

SQL

```sql
-- Buscar mensagens similares
SELECT id, content, 1 - (embedding <=> query_embedding) as similarity
FROM conversation_messages
WHERE user_id = 'user_123'
AND 1 - (embedding <=> query_embedding) > 0.6
```

```
ORDER BY similarity DESC
LIMIT 5;
```

🛠️ Troubleshooting

Erro: "GEMINI_API_KEY não configurada"

```bash
# Verificar variável
echo $GEMINI_API_KEY

# Se vazio, adicionar ao .env
GEMINI_API_KEY=sua_chave_aqui
```

Erro: "Conexão com Supabase falhou"

```bash
# Verificar credenciais
echo $SUPABASE_URL
echo $SUPABASE_KEY

# Testar conexão
python3 -c "from supabase import create_client; create_client('$SUPABASE_URL',
```

Busca retorna resultados vazios

- Aumentar `threshold` (padrão: 0.6 para Arth, 0.5 para ChefIA)
- Verificar se há dados: `SELECT COUNT(*) FROM recipes`
- Verificar embeddings: `SELECT COUNT(*) FROM recipes WHERE embedding IS NOT NULL`

Performance lenta

```sql
-- Verificar índices
SELECT * FROM pg_indexes WHERE tablename = 'recipes';

-- Otimizar tabelas
```

```
VACUUM ANALYZE recipes;
VACUUM ANALYZE conversation_messages;
```

📚 Documentação Completa

- **Arth Executive**: `/home/ubuntu/arth_embedding/README.md`
- **ChefIA**: `/home/ubuntu/chefia_embedding/README.md`

🔗 Referências

- Google Generative AI SDK
- Supabase pgvector
- Gemini Embedding Models

💡 Próximos Passos

- [ ] Implementar cache de embeddings
- [ ] Adicionar suporte a múltiplos idiomas
- [ ] Criar dashboard de analytics
- [ ] Implementar export de dados
- [ ] Adicionar suporte a embeddings em batch

🤝 Suporte

Para dúvidas:

1. Verificar README.md específico do projeto
2. Consultar documentação do Supabase
3. Revisar logs em `*.log`
4. Abrir issue no GitHub

Criado em: 22 de março de 2026

Versão: 1.0.0

Suporte: Gemini Embedding 2 + Supabase pgvector